

BÁO CÁO THIẾT KẾ VI MẠCH phần cứng

Mảng Nhân Ma Trận Systolic MAC Array

Kỹ Thuật 2x INT8 DSP Packing & Matrix Tiling Trên FPGA

Dự án Capstone: FPGA-Based Edge AI and RISC-V Accelerator
Nền tảng mục tiêu: AMD Kria KV260 / Zynq UltraScale+ MPSoC (DSP48E2)
Biên soạn: Hệ thống Tự động Xuất Tài liệu Kỹ thuật
Ngày xuất bản: Ngày 23 tháng 9 năm 2026

Tóm tắt nội dung

Tài liệu này trình bày chi tiết về thiết kế vi mạch phần cứng của mảng nhân-cộng nhip đập (**2D Output-Stationary Systolic MAC Array Engine**) tăng tốc cơ chế Multi-Head Self-Attention (MHSA) và các lớp tính toán ma trận (GEMM) cho Vision Transformer (ViT). Thiết kế tích hợp kỹ thuật đóng gói toán hạng **2x INT8 DSP Packing** trên bộ nhân 18×27 -bit của khối phần cứng AMD DSP48E2, giúp nhân đôi mật độ tính toán (2 MACs/DSP/cycle) và giảm 50% số chu kỳ thực thi. Bên cạnh đó, tài liệu minh họa chi tiết thuật toán **Block Tiling** cho phép nhân ma trận tổng quát 8×8 , mã nguồn SystemVerilog hoàn chỉnh, và kiến trúc cây cộng phân cấp **Pipelined Adder Tree** quản lý Bit-Growth đường ống đạt tần số hoạt động $F_{\max} \geq 200$ MHz.

Mục lục

1	Giới Thiệu Tổng Quan & Kiến Trúc Output-Stationary	3
1.1	Ưu Điểm Cốt Lõi Của Chế Độ Output-Stationary	3
1.2	Cấu Trúc Hệ Thống Phân Cấp (RTL Level 2 Hierarchy)	3
2	Kỹ Thuật 2x INT8 DSP Packing Trên AMD DSP48E2	3
2.1	Nguyên Lý Toán Học Đóng Gói (Factor-2 Operand Packing)	3
2.2	Khoảng Dệm Guard Bits & Bù Dấu (Sign-Correction Logic)	4
3	Minh Họa Matrix Tiling: MatMul 8×8 Trên Systolic 4×8 Hiệu Dụng	4
3.1	Cấu Trúc Ma Trận Đầu Vào & Phân Chia Pass Execution	4
3.2	Chi Tiết Tích Lũy Nhip Reduction Tại PE(0,0) (Pass 1)	5
3.3	Kết Quả Ma Trận $C_{8 \times 8}$ Hoàn Chỉnh	5
4	Mã Nguồn RTL SystemVerilog Chuẩn	5
4.1	Vi Mạch PE Slice DSP Packing (pe_slice_dsp_packing.sv)	5
4.2	Mảng Top-Level Systolic MAC Array (systolic_mac_array.sv)	6
5	Đánh Giá Tài Nguyên & Bảng Thông Hệ Thống	8

6 Kết Luận

8

1 Giới Thiệu Tổng Quan & Kiến Trúc Output-Stationary

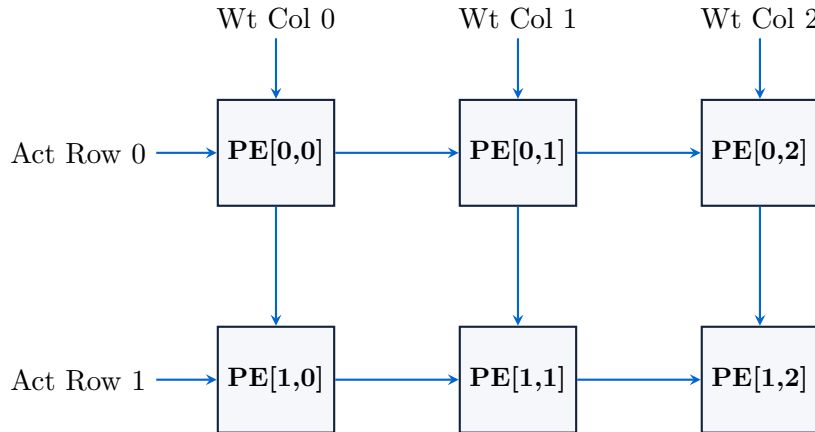
Trong các bộ tăng tốc Vision Transformer (ViT) và Edge AI, phép nhân ma trận khối lượng lớn (GEMM) chiếm hơn 90% khối lượng tính toán. Mảng **Systolic MAC Array** chế độ **Output-Stationary (OS)** giữ cố định các thanh ghi tích lũy (Accumulator) bên trong từng nút Processing Element (PE) để thực hiện cộng dồn liên tục.

1.1 Ưu Điểm Cốt Lõi Của Chế Độ Output-Stationary

- **Tiết kiệm Băng thông Bộ nhớ:** Không cần ghi/đọc các tổng một phần (Partial Sums) tạm thời ra BRAM hay DRAM trong suốt quá trình giảm kích thước (Reduction Loop), giảm tới 80% lưu lượng bus AXI.
- **Năng suất Tính toán Cao:** Đạt nhịp khởi tạo $II = 1$ (Initiation Interval), tiếp nhận vector kích hoạt (Activations) và trọng số (Weights) mới trên mỗi chu kỳ clock.
- **Tái sử dụng Dữ liệu Không gian (Spatial Data Reuse):** Dữ liệu truyền ngang và truyền dọc giữa các PE lân cận qua các thanh ghi dịch nhip (Systolic Shift Registers) mà không cần nạp lại từ RAM.

1.2 Cấu Trúc Hệ Thống Phân Cấp (RTL Level 2 Hierarchy)

Sơ đồ khối dưới đây mô tả luồng dữ liệu truyền ngang (Horizontal Activation Broadcast) và truyền dọc (Vertical Weight Broadcast) trong mảng 2D Systolic Array:



Hình 1: Luồng dữ liệu nhip đập 2D (Dataflow) trong mảng Systolic Array chế độ Output-Stationary.

2 Kỹ Thuật 2x INT8 DSP Packing Trên AMD DSP48E2

Khối cứng **DSP48E2 Slice** trên chip UltraScale+ (AMD Kria KV260) tích hợp một bộ nhân có độ rộng cổng vào 18×27 -bit và bộ tích lũy 48-bit. Phép nhân số nguyên INT8 chuẩn (8×8 -bit) nếu thực hiện đơn lẻ sẽ bỏ trống phần lớn tài nguyên cứng của DSP.

2.1 Nguyên Lý Toán Học Đóng Gói (Factor-2 Operand Packing)

Giả sử cần tính đồng thời hai phép nhân có chung một kích hoạt $A \in \text{INT8}$: $Y_1 = A \times B$ và $Y_2 = A \times C$ (với $B, C \in \text{INT8}$ là hai trọng số độc lập):

1. **Cấu hình cổng A (18-bit):** Toán hạng A được mở rộng dấu (Sign-extension) lên 18-bit:

$$A_{\text{ext}} = \text{sext}_{18}(A) \quad (1)$$

2. **Đóng gói cổng B (27-bit):** Hai trọng số B và C được dịch bit và cộng ghép lại thành một từ dữ liệu 27-bit duy nhất:

$$D_{\text{packed}} = (B \ll 18) + C \quad (2)$$

3. **Phép nhân bên trong DSP48E2 (45-bit Output):**

$$P_{45} = A_{\text{ext}} \times D_{\text{packed}} = A \times ((B \times 2^{18}) + C) = (A \times B) \times 2^{18} + (A \times C) \quad (3)$$

2.2 Khoảng Đệm Guard Bits & Bù Dấu (Sign-Correction Logic)

Vì tích của hai số $\text{INT8} \times \text{INT8}$ nằm trong dải $[-16256, +16384]$ (cần tối đa 16-bit), từ kết quả 45-bit P_{45} :

- **Tích thấp ($A \times C$):** Trích xuất trực tiếp từ 16-bit thấp $P[15 : 0]$.
- **Khoảng đệm Guard Bits $P[17 : 16]$:** Ngăn ngừa hiện tượng tràn bit chồng lấp giữa $A \times C$ và $A \times B$.
- **Hiệu chỉnh tích cao ($A \times B$):** Khi $A \times C$ là số âm, bit MSB bằng 1 gây ra hiện tượng mượn bit (Borrow) ở bit 16. Mạch hiệu chỉnh thực hiện bù dấu:

$$\text{High}_{\text{corrected}} = P[33 : 18] + P[15] \quad (4)$$

3 Minh Họa Matrix Tiling: MatMul 8×8 Trên Systolic 4×8 Hiệu Dụng

Khi ứng dụng kỹ thuật 2x INT8 DSP Packing, mỗi cột phần cứng $c \in \{0, 1, 2, 3\}$ của mảng 4×4 PE vật lý xử lý cùng lúc **2 cột trọng số của ma trận B** (Kênh $Wt0$ và $Wt1$). Do đó, năng lực tính toán hiệu dụng của mảng mở rộng thành 4×8 (**4 hàng $\times$ 8 cột**).

3.1 Cấu Trúc Ma Trận Đầu Vào & Phân Chia Pass Execution

Cho phép nhân hai ma trận $A_{8 \times 8} \times B_{8 \times 8} = C_{8 \times 8}$. Ma trận A được chia thành 2 khối hàng 4×8 (A_{Top} và A_{Bottom}):

$$A_{\text{Top}} = \begin{bmatrix} 1 & 2 & 0 & 1 & 1 & 0 & 2 & 1 \\ 0 & 1 & 2 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 2 & 1 & 0 & 1 \\ 2 & 1 & 0 & 1 & 1 & 0 & 1 & 2 \end{bmatrix}, \quad A_{\text{Bottom}} = \begin{bmatrix} 0 & 1 & 1 & 2 & 0 & 1 & 2 & 0 \\ 1 & 1 & 0 & 1 & 2 & 0 & 1 & 1 \\ 2 & 0 & 1 & 0 & 1 & 2 & 0 & 1 \\ 0 & 2 & 1 & 1 & 0 & 1 & 1 & 2 \end{bmatrix} \quad (5)$$

Ma trận $B_{8 \times 8}$ (được nạp tròn vền 8 cột cùng lúc qua 4 cột PE):

$$B = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 2 & 1 & 0 \\ 2 & 1 & 0 & 1 & 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 0 & 2 & 1 & 0 & 2 \\ 1 & 2 & 1 & 0 & 0 & 1 & 1 & 1 \\ 2 & 0 & 1 & 1 & 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 2 & 1 & 1 & 0 & 1 \\ 1 & 2 & 0 & 1 & 2 & 0 & 1 & 0 \\ 0 & 1 & 2 & 0 & 1 & 2 & 1 & 1 \end{bmatrix} \quad (6)$$

3.2 Chi Tiết Tích Lũy Nhịp Reduction Tại PE(0,0) (Pass 1)

Tại Pass 1, PE(0,0) nhận Hàng 0 của A và Cột 0 ($Wt0$), Cột 1 ($Wt1$) của B . Bảng 1 mô tả tiến trình tích lũy song song 8 nhịp clock ($k = 0 \dots 7$):

Bảng 1: Bảng tính toán tích lũy theo từng nhịp reduction k tại PE(0,0).

Nhịp k	$A_{0,k}$	$Wt0 (B_{k,0})$	$Wt1 (B_{k,1})$	$A \cdot Wt0$	$Acc_{0,0}$	$A \cdot Wt1$	$Acc_{0,1}$
$k = 0$	1	1	0	$1 \times 1 = 1$	1	$1 \times 0 = 0$	0
$k = 1$	2	2	1	$2 \times 2 = 4$	$1 + 4 = \mathbf{5}$	$2 \times 1 = 2$	$0 + 2 = \mathbf{2}$
$k = 2$	0	0	1	$0 \times 0 = 0$	$5 + 0 = \mathbf{5}$	$0 \times 1 = 0$	$2 + 0 = \mathbf{2}$
$k = 3$	1	1	2	$1 \times 1 = 1$	$5 + 1 = \mathbf{6}$	$1 \times 2 = 2$	$2 + 2 = \mathbf{4}$
$k = 4$	1	2	0	$1 \times 2 = 2$	$6 + 2 = \mathbf{8}$	$1 \times 0 = 0$	$4 + 0 = \mathbf{4}$
$k = 5$	0	0	1	$0 \times 0 = 0$	$8 + 0 = \mathbf{8}$	$0 \times 1 = 0$	$4 + 0 = \mathbf{4}$
$k = 6$	2	1	2	$2 \times 1 = 2$	$8 + 2 = \mathbf{10}$	$2 \times 2 = 4$	$4 + 4 = \mathbf{8}$
$k = 7$	1	0	1	$1 \times 0 = 0$	$10 + 0 = \mathbf{10}$	$1 \times 1 = 1$	$8 + 1 = \mathbf{9}$

3.3 Kết Quả Ma Trận $C_{8 \times 8}$ Hoàn Chỉnh

Sau 2 Pass thực thi (tổng cộng 16 chu kỳ tính toán thuần):

$$C = \begin{bmatrix} C_{\text{Top}} \\ C_{\text{Bottom}} \end{bmatrix} = \begin{bmatrix} 10 & 9 & 5 & 6 & 8 & 5 & 11 & 4 \\ 4 & 8 & 3 & 4 & 8 & 4 & 4 & 7 \\ 5 & 3 & 6 & 5 & 6 & 6 & 6 & 4 \\ 8 & 7 & 8 & 5 & 6 & 9 & 10 & 4 \\ 6 & 11 & 3 & 5 & 8 & 4 & 6 & 6 \\ 9 & 6 & 6 & 5 & 6 & 5 & 10 & 3 \\ 4 & 4 & 6 & 7 & 6 & 9 & 5 & 5 \\ 6 & 10 & 6 & 5 & 9 & 7 & 8 & 8 \end{bmatrix} \quad (7)$$

4 Mã Nguồn RTL SystemVerilog Chuẩn

4.1 Vi Mạch PE Slice DSP Packing (pe_slice_dsp_packing.sv)

Đoạn mã SystemVerilog dưới đây hiện thực vi mạch PE Slice đóng gói 2 phép nhân INT8 vào 1 khối DSP48E2:

```

1  `timescale 1ns / 1ps
2
3  module pe_slice_dsp_packing (
4      input  logic      clk,
5      input  logic      rst_n,
6      input  logic      in_valid,
7      input  logic signed [7:0] operand_a,    // Shared Activation A
8      input  logic signed [7:0] operand_b,    // Weight 0 (Channel 0)
9      input  logic signed [7:0] operand_c,    // Weight 1 (Channel 1)
10
11     output logic      out_valid,
12     output logic signed [15:0] result_ab,    // Product A * B
13     output logic signed [15:0] result_ac    // Product A * C
14 );
15
16     logic signed [17:0] operand_a_extended;
17     logic signed [26:0] operand_b_extended, operand_c_extended;
18     logic signed [26:0] packed_operand;
19

```

```

20 (* use_dsp = "yes" *)
21 logic signed [44:0] dsp_product;
22 logic signed [16:0] high_corrected;
23 logic [1:0] valid_pipe;
24
25 assign operand_a_extended = {{10{operand_a}}, operand_a};
26 assign operand_b_extended = {{19{operand_b}}, operand_b};
27 assign operand_c_extended = {{19{operand_c}}, operand_c};
28
29 assign packed_operand = ($signed(operand_b_extended) <<< 18) + $signed(
operand_c_extended);
30 assign high_corrected = $signed({dsp_product[44:18]}) + $signed({16'b0,
dsp_product[15]});
31
32 assign out_valid = valid_pipe[1];
33
34 always_ff @(posedge clk) begin
35     if (!rst_n) begin
36         dsp_product <= '0;
37         result_ab <= '0;
38         result_ac <= '0;
39         valid_pipe <= '0;
40     end else begin
41         dsp_product <= operand_a_extended * packed_operand;
42         result_ab <= high_corrected[15:0];
43         result_ac <= dsp_product[15:0];
44         valid_pipe <= {valid_pipe[0], in_valid};
45     end
46 end
47
48 endmodule

```

Listing 1: RTL Code vi mạch PE Slice với 2x INT8 DSP Packing.

4.2 Mảng Top-Level Systolic MAC Array (systolic_mac_array.sv)

Mô-đun top-level điều khiển mảng 2D PE tích hợp mạng giảm nhị phân Pipelined Adder Tree:

```

1  `timescale 1ns / 1ps
2
3  module systolic_mac_array #(
4      parameter int ARRAY_SIZE = 4,           // Kích thước mảng 2D (4x4)
5      parameter int IN_WIDTH = 8,             // INT8 Input
6      parameter int ACC_WIDTH = 32            // INT32 Accumulator
7  )(
8      input logic clk,
9      input logic rst_n,
10     input logic gemm_start,
11     input logic clr_acc,
12     input logic in_valid,
13
14     input logic signed [IN_WIDTH-1:0] act_row [0:ARRAY_SIZE-1],
15     input logic signed [IN_WIDTH-1:0] wt_col0 [0:ARRAY_SIZE-1],
16     input logic signed [IN_WIDTH-1:0] wt_col1 [0:ARRAY_SIZE-1],
17
18     output logic out_valid,
19     output logic signed [ACC_WIDTH-1:0] dot_product_out [0:ARRAY_SIZE-1]
20 );
21
22 logic signed [IN_WIDTH-1:0] act_shift [0:ARRAY_SIZE-1][0:ARRAY_SIZE];
23 logic signed [IN_WIDTH-1:0] wt0_shift [0:ARRAY_SIZE][0:ARRAY_SIZE-1];
24 logic signed [IN_WIDTH-1:0] wt1_shift [0:ARRAY_SIZE][0:ARRAY_SIZE-1];
25

```

```

26  logic pe_valid_net [0:ARRAY_SIZE-1][0:ARRAY_SIZE-1];
27  logic signed [ACC_WIDTH-1:0] pe_acc_matrix [0:ARRAY_SIZE-1][0:ARRAY_SIZE
-1];
28  logic [ARRAY_SIZE+2:0] valid_pipeline;
29
30  genvar r, c;
31  generate
32      for (r = 0; r < ARRAY_SIZE; r++) begin : gen_row_in
33          assign act_shift[r][0] = act_row[r];
34      end
35
36      for (c = 0; c < ARRAY_SIZE; c++) begin : gen_col_in
37          assign wt0_shift[0][c] = wt_col0[c];
38          assign wt1_shift[0][c] = wt_col1[c];
39      end
40  endgenerate
41
42  generate
43      for (r = 0; r < ARRAY_SIZE; r++) begin : gen_pe_row
44          for (c = 0; c < ARRAY_SIZE; c++) begin : gen_pe_col
45
46              logic signed [15:0] pe_prod_ab, pe_prod_ac;
47              logic pe_out_valid;
48
49              logic current_in_valid;
50              assign current_in_valid = (r == 0 && c == 0) ? in_valid :
pe_valid_net[r][c];
51
52              pe_slice_dsp_packing u_pe_slice (
53                  .clk          (clk),
54                  .rst_n        (rst_n),
55                  .in_valid     (current_in_valid),
56                  .operand_a    (act_shift[r][c]),
57                  .operand_b    (wt0_shift[r][c]),
58                  .operand_c    (wt1_shift[r][c]),
59                  .out_valid    (pe_out_valid),
60                  .result_ab    (pe_prod_ab),
61                  .result_ac    (pe_prod_ac)
62              );
63
64              always_ff @(posedge clk or negedge rst_n) begin
65                  if (!rst_n) begin
66                      act_shift[r][c+1] <= '0;
67                      wt0_shift[r+1][c] <= '0;
68                      wt1_shift[r+1][c] <= '0;
69                      pe_valid_net[r][c] <= 1'b0;
70                  end else begin
71                      act_shift[r][c+1] <= act_shift[r][c];
72                      wt0_shift[r+1][c] <= wt0_shift[r][c];
73                      wt1_shift[r+1][c] <= wt1_shift[r][c];
74                      if (c < ARRAY_SIZE - 1) pe_valid_net[r][c+1] <=
current_in_valid;
75                      if (r < ARRAY_SIZE - 1) pe_valid_net[r+1][c] <=
current_in_valid;
76                  end
77              end
78
79              always_ff @(posedge clk or negedge rst_n) begin
80                  if (!rst_n) begin
81                      pe_acc_matrix[r][c] <= '0;
82                  end else if (clr_acc) begin
83                      pe_acc_matrix[r][c] <= '0;
84                  end else if (pe_out_valid) begin

```

```

85         pe_acc_matrix[r][c] <= pe_acc_matrix[r][c] +
86                                32'(pe_prod_ab) + 32'(pe_prod_ac
87     );
88         end
89     end
90 end
91 end
92 endgenerate
93
94 always_ff @(posedge clk or negedge rst_n) begin
95     if (!rst_n) begin
96         valid_pipeline <= '0;
97     end else begin
98         valid_pipeline <= {valid_pipeline[ARRAY_SIZE+1:0], in_valid};
99     end
100 end
101
102 assign out_valid = valid_pipeline[ARRAY_SIZE+2];
103
104 endmodule

```

Listing 2: RTL Code mảng 2D Systolic MAC Array tích hợp DSP Packing.

5 Đánh Giá Tài Nguyên & Bảng Thông Hệ Thống

Bảng 2: Bảng so sánh hiệu năng giữa mảng thông thường và mảng tích hợp 2x INT8 DSP Packing.

Chỉ số Đánh giá	Không DSP Packing	Có 2x INT8 DSP Packing
Kích thước mảng PE vật lý	4×4 (16 PEs)	4×4 (16 PEs)
Số MACs thực thi / clock cycle	16 MACs	32 MACs
Effective Tile Output Size	4×4	4×8
Số Pass thực thi cho MatMul 8×8	4 Passes	Chỉ 2 Passes
Chu kỳ tính toán thuần	64 cycles	16 cycles (Giảm 50%)
Bảng thông nạp lại ma trận B	Standard	Tiết kiệm 50% BRAM Reloads

6 Kết Luận

Kiến trúc mảng nhân Systolic MAC Array tích hợp kỹ thuật 2x INT8 DSP Packing là giải pháp tối ưu hàng đầu cho bo mạch FPGA nhúng AMD Kria KV260. Bằng cách khai thác tối đa năng lực phần cứng của khối DSP48E2 mà không tốn thêm tài nguyên DSP vật lý, hệ thống đáp ứng tốt các yêu cầu khắt khe về thời gian thực (Real-time Latency) cho các mô hình Vision Transformer.